

Analysis of Sequence Modeling

Shivanshu Kumar
v1.0

Abstract

This report aims to capture the evolution of Machine Learning Techniques in Language Modeling.

This document further takes a detour to studying the utility of transformers for vision and informatics related tasks, which are not inherently sequential.

Overall this document tries to study:

1. Language modeling as a problem of Artificial Intelligence and methods to make it efficient.
2. Reasoning paradigms in and outside of language modeling

Contents

Abstract	i
I Evolution of Sequence Models	3
1 Generation 1 : $O(1)$ hidden state	4
1.1 RNNs	4
1.2 LSTMs and GRUs	6
2 Generation 2 : Transformers	7
2.1 Encoder-Decoder Architecture	7
2.2 Decoder Only Architecture	7
2.3 State of the Art Language Models	9
2.3.1 GPT	9
2.3.2 BERT	9
2.3.3 Qwen	10
2.3.4 Mistral	10
2.3.5 Llama	10
2.3.6 Phi	10
2.3.7 DeepSeek	10
3 Efficiency	12
3.1 Distillation	12
3.2 Sparsity through MOE	13
3.3 Efficient Implementation	13
3.4 Eliminating Redundancy	14
3.4.1 Pruning	14
3.4.2 KV-sharing and elimination	15
3.4.3 Quantization	15
3.5 Some other Techniques	15
3.5.1 Gradient Checkpointing	15
3.5.2 Choosing the optimizer	15
4 Generation 3	16
4.1 State Space Models	16
4.1.1 Linear Attention	16
4.1.2 Linformer	18
4.1.3 RWKV	18
4.1.4 Reformer	18
4.1.5 RetNet	18
4.1.6 Mamba	18
4.1.7 Gated-Linear Attention	18

4.2	Diffusion Language Models	19
4.3	Energy based Models	20
4.4	Hyena architectures	21
4.5	Hybrid Models	21
4.6	Notion of Heirarchical Thinking	22
4.7	Scaling laws for Language models	22
II	LLMs as Chat-bots	23
5	Post-training	24
5.1	RLHF : Reinforcement Learning from Human Feedback	24
5.2	Instruction Fine-tuning	24
6	AGI and Reasoning	26
6.1	How are reasoning models like GPT-o, Claude Thinking etc trained?	26
6.1.1	GPT-o series	26
6.2	ARC-AGI Benchmarks	26
6.3	Some other interesting benchmarks	26
6.4	Current Reasoning Landscape	26
6.5	Hierarchical Reasoning Model	27
6.6	Scientist Language Models	27
6.7	Monte Carlo Search Method	28
6.8	Reinforcement Learning	28
6.9	Latent Reasoning	29
7	Safety in AI	30
8	Interpretability insights	31
8.1	Random Stuff	31
8.2	Optimizers	31
8.3	Transformer++ recipe	31
8.4	Datasets	31
8.5	benchmarks	31
8.6	Theory of consistent information flow	32
8.7	Theory of iterative inference	33
9	Multimodality - Transformers as efficient parameitrizers	34

Part I

Evolution of Sequence Models

Generation 1 : $O(1)$ hidden state

The task of sequence modeling involves producing outputs from a sequence of inputs of the same type - this output could in itself be a sequence. In the latter case the problem is termed as "Sequence-to-Sequence Modeling".

The problem of language modeling (generation) falls under this category since the task of auto-completion or conversation involves taking a text input of arbitrary size and then outputting another sequence of arbitrary size (length). Another feature of language is "context". Each sub-input carries its own context which might change drastically if some part of the input is ignored, hence to generate the output sequence the entire input should be kept in mind.

A common model to accomplish this is the **auto-regressive paradigm** wherein, given an input *prompt* $t_1, t_2 \dots t_k$ of words, the model is aimed at producing

$$P(t_{k+1} | t_1 \dots t_k)$$

Note that framing the problem in such a way makes it recursive, i.e., to generate the next word¹ we set input = $t_1, t_2 \dots t_k$ and repeat the process n times. Hence a sequence $t_{k+1} \dots t_{k+n}$ is obtained, which is the output of the model (Language generator). The underlying sequence-to-next-token module is a neural network.

Now given this auto-regressive framework where we can have arbitrary inputs of any length and all the tokens in the sequence contribute to the output, including the ones that are generated – one simple way is to store a "hidden state" representing the dynamic state of the generated sequence, where all the tokens add their contributions as they are generated.

Observe that the expression

$$\left(\prod_{i=2}^n P(t_i | t_{i-1} \dots t_1) \right) \times P(t_1) = P(t_1 \dots t_n)$$

So iterating the token generation conditioned over the previous sub-sequence is essentially computing the joint probability of the entire sequence!

1.1 RNNs

The most vanilla model as mentioned above is a Recurrent neural network - named so, since the computation repeats over "steps". More formally, the Neural Network maintains a hidden state s_i at every time step i (corresponding to the i^{th} token in the sequence. With every incoming input token x_i , the state is updated as follows.

$$\begin{aligned} s_i &= (Ux_i + Ws_{i-1} + b) \\ y_i &= \sigma(Vs_i + c) \end{aligned} \tag{1.1}$$

¹in practice the granularity of language models is a sub-word aka a token

Note that prior to any input the state s_0 could be set randomly or learned along with the weights U , V and W . y_i represents the output at time step i .

Suppose there is input sequence $x_1, x_2, \dots, x_k$, the outputs $y_1, \dots, y_k$ can be ignored and for $i > k$ we can set $x_i = y_{i-1}$, which clearly fits our auto regressive language generation paradigm.

Typical Neural Networks are trained via Gradient based Backpropagation algorithms - which require gradient computation of the defined loss function with respect to the weights of the network.

In order to compute loss on the entire sequence: we take the binary cross entropy loss at each time step, which is the probability of the predicted token:

$$L_t = -\log(P(y_t|x_1\dots x_t))$$

. The total loss being

$$L = \sum L_i$$

the sequence is typically truncated unless the model predicts $\langle eos \rangle$ (End of Sequence) on its own. The derivative now becomes

$$\frac{\partial L}{\partial W} = \sum \frac{\partial L_i}{\partial W}$$

Note that the gradient computation is tricky, since for L_j ,

$$\frac{\partial L_j}{\partial W} = \frac{\partial L_j}{\partial s_j} \frac{\partial s_j}{\partial W}$$

this comes from the chain rule – since L_j is a direct function of only s_j . So the computational dependencies of L_j on previous tokens is through state update rules of s_j which is captured in $s_j = Us_{j-1} + Wx_j + b$. Note that since s_{j-1} also depends on U , W , the gradient computation is non-trivial and involves a **back-propagation through time**. We employ the multiplication rule of derivatives to obtain :

$$\frac{\partial s_j}{\partial W} = \underbrace{\frac{\partial^+ s_j}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_j}{\partial s_{j-1}} \frac{\partial s_{j-1}}{\partial W}}_{\text{implicit}}$$

For $j = 4$, the computation is unrolled and shown (Credits [Prof. Mitesh Khapra's slides](#))

$$\begin{aligned} &= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3} \left[\underbrace{\frac{\partial^+ s_3}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial W}}_{\text{implicit}} \right] \\ &= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial s_2} \left[\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial W} \right] \\ &= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial^+ s_2}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \left[\frac{\partial^+ s_1}{\partial W} \right] \end{aligned}$$

The above algorithm is extremely problematic for the same reason as very deep neural networks are, too many multiplications in a term leading to **vanishing or exploding** gradients.

This, along with the fact that RNNs weigh each token equally in the "context vector" which is essentially a 256 – 512 dimensional vector leads to context pollution in practice, making them unfit for language modeling. Therefore we introduce **Gating mechanisms** to focus on the "important" tokens.

1.2 LSTMs and GRUs

In a **Long Short Term Memory** – do selective read and write into the hidden state by using a learned binary gate $o_t \in \{0, 1\}^n$ which masks the state for updation i.e., and a gated hidden output $h_t = o_t \odot s_t$ is used.

To update s_t , we use i_t, f_t which are fancily called the input and forget gates, updated using the previous state and current input for each time step:

$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

Selective read :

$$s'_t = U h_{t-1} + W x_t + b$$

Selective write :

$$s''_t = i_t \odot s'_t$$

Selective forget (I never got why this exists):

$$s_t = s''_t + f_t \odot s_{t-1}$$

A variant is called the **Gated Recurrent unit** which eliminates f_t and uses $1 - i_t$ instead, and uses s_{t-1} directly for updating the gates :

$$o_t = \sigma(W_o h_{s-1} + U_o x_t + b_o)$$

$$i_t = \sigma(W_i h_{s-1} + U_o x_t + b_i)$$

$$s_t = i_t \odot s'_t + (\mathbb{I} - i_t) \odot s_{t-1}$$

Note that the gradient flow in this case is gate-controlled and any time step not contributing is by-passed.

Problem with this is, once something is bypassed it is bypassed forever – for long context scenarios, say a word that is relevant for another is far behind in the sentence (Source : Google Gemini) :

”The highly intellectual and renowned **professor**, who had authored numerous books and was admired by countless students, finally revealed **his** groundbreaking theory on the universe’s origin.”

the context for ”his” is far behind sitting with the word ”professor” – this typically happens in a lot of sequence modeling tasks. The fundamental limitation with these models is that they store sequence level information in constant space and even the state-of-the-arts were unable to support more than hundreds of tokens. Transformers, as discussed in the next chapter solve this problem by storing the information corresponding to each of the tokens separately.

Generation 2 : Transformers

This generation of sequence-to-sequence models store entire context , i.e., $O(n)$ in the size of the sequence (n). Introduced in the seminal paper : Vaswani et al. 2017, Transformers have become the de-facto for Large Language Models, which are conversational agents built on top of decoder-only transformers, by fine-tuning them on instruction and conversation data. What we end up getting is an encoder decoder model, which could have two different modalities as input and output as long as the hidden state dimensions are the same.

Attention MechanismAs mentioned before, for every step of generation in the decoder model, the context given is constant-size – you can choose to pass the state information from the encoder model at every time step (by concatenating with x_t) in the decoder model, but the point is, the relevance of tokens in the input to the tokens in the generated sequence is not uniform. Attention solves this problem by appropriately weighing the input tokens to encoder model, at each step of generation.

2.1 Encoder-Decoder Architecture

The original transformer architecture follows an encoder-decoder design for sequence-to-sequence tasks. The encoder processes the entire input sequence using full (bidirectional) self-attention to create context-aware vector representations for each token. These representations are then passed to the decoder. During decoding, output tokens are generated one-by-one in an auto-regressive fashion. At each decoding step, the decoder receives the previously generated tokens and attends to both the encoded input sequence (via encoder-decoder cross-attention) and the previous decoder outputs. The decoder’s self-attention uses a causal mask (Masked Multi-Headed Self-Attention) so each token only attends to previous tokens, preventing access to future tokens during training. (See figure 2.1)

2.2 Decoder Only Architecture

Radford and Narasimhan 2018 introduced decoder-only transformers as a more-suited architecture for Natural-Language Generation. Scaling such models up to hundreds to billions of parameters gives us Large Language Models as we know them today. During training, loss is computed using cross-entropy loss function on the output probability vector $\hat{y}_t \in \mathbb{R}^V$ and the one-hot label for that position $y_t \in \{0,1\}^V$.

$$L = -\frac{1}{T} \sum_{t=1}^T \sum_{i=1}^V y_{t,i} \log \hat{y}_{t,i}$$

Since y_t is a one-hot vector this essentially is negative log-probability for the label token as predicted by our model.

$$L = -\frac{1}{T} \sum_{t=1}^T y_{t,i_t} \log \hat{y}_{t,i_t}$$

See Figure 2.2

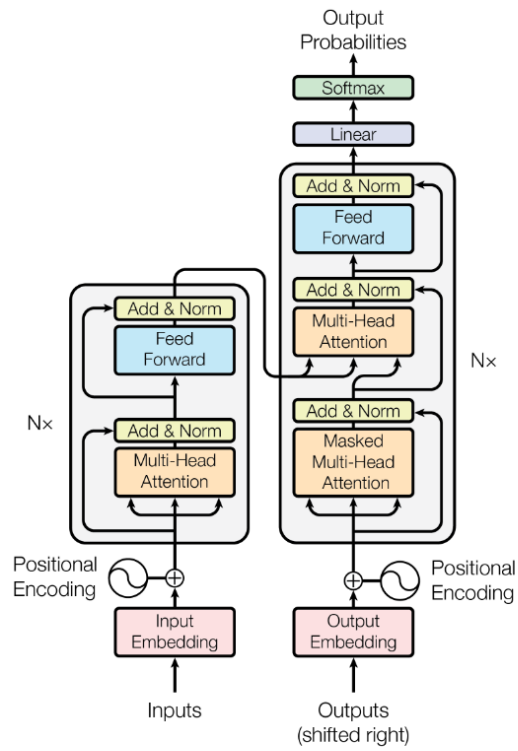


Figure 1: The Transformer - model architecture.

Figure 2.1: Encoder-Decoder Transformer

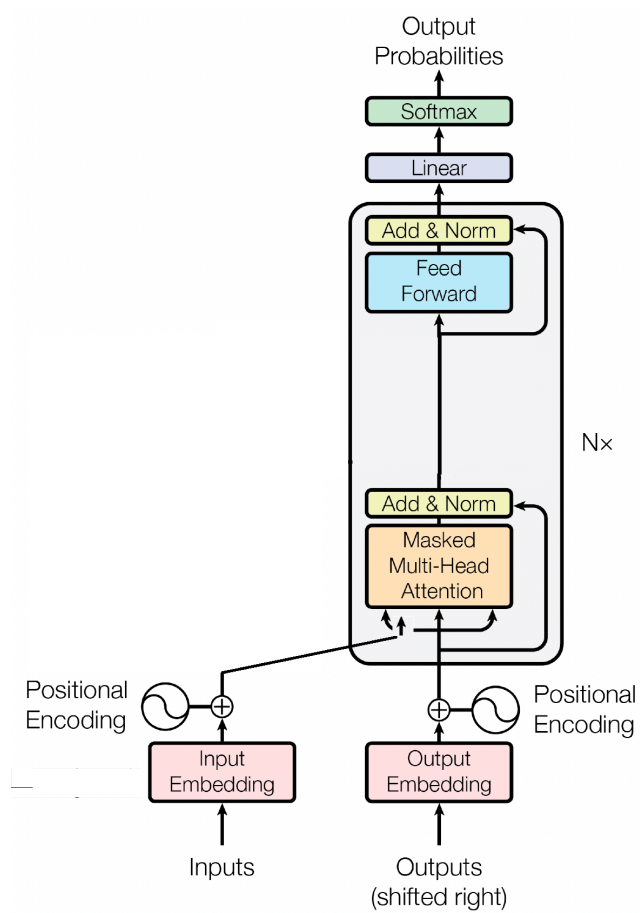


Figure 2.2: Decoder-only Transformers

2.3 State of the Art Language Models

2.3.1 GPT

Generative Pre-trained Transformers introduced by Radford and Narasimhan, train a Language Model that is "pre-trained" on a self-supervised objective and further fine-tune (supervised) on labeled datasets for the task at hand, without changing the backbone model architecture. Utilize a decoder-only transformer architecture trained on a language generation objective. Key insight is that transformers trained on large unlabeled corpora can transfer knowledge across various tasks with little fine-tuning.

Model : GPT-2 was trained with dropout

* **Google PaLM** is a 540 B parameter model (released before Gemini was introduced for multi-modality) with the aim for scaling params. Demonstrate efficient deployment on a large number of TPUs using "PathWay" technique designed for asynchronous distribution of compute. Use Parallel architecture : $y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$. Also utilize MQA N. Shazeer [2019](#).

* **Pythia** (Biderman et al. [2023](#)) also uses parallel MLP and attention.

2.3.2 BERT

Bidirectional Encoder Representations Source: Devlin et al. [2019](#).

- Encoder-only model used for **Language understanding** or *Representation Learning*
- Jointly conditions both through the left and right by introducing bi-directional context and minimal architectural changes for downstream application (like GPT)
- As opposed to the auto-regressive models, use Masked Language Modeling - replace some tokens randomly with a mask and predict those tokens
- Pre-training objective: using two bidirectional models does not help : since when predicting the $t + 1^{th}$ token, t^{th} token has context from it, this becomes trivial for the model to "peek" into the answer before prediction.
- All the tokens self-attend to each other – refining representations through the layers and for final prediction etc, use the [CLS] token's final representation.
- For fine-tuning send the input-output pairs concatenated which is essentially cross-attention between the two!
- For measuring performance on say GLUE: use final representation of [CLS] and softmax through it
- ROBERTA Y. Liu et al. [2019](#) improved BERT by (1) applying dynamic instead of static masking, (2) evaluating different model input formats and the disregarding the necessity of Next Sentence Prediction (NSP)(3) experimenting with larger batch sizes and Applying BPE text encoding, training with more data and performing optimal hyperparameter optimization.

2.3.3 Qwen

QwQ : Qwen with questions – verification through questioning

extremely efficient and high performant models at that size

2.3.4 Mistral

Training is data compression!

much better performance due to excellent data and extensive training beyond chinchilla scaling which helped them for better inference costs

2.3.5 Llama

Vicuna is the SFT version of Llama, similar to Models like Gemini-2.5 and GPT-4 can handle multi-modal data through a single Transformer backbone. MobileLLMs (Z. Liu et al. 2024) utilize a deep but thin architecture along with Grouped-Query-Attention(Ainslie et al. 2023), built on top of LLaMA . Use about 1T tokens for training all models from size 125M to 1.5B with 30 to 55 layers. **Large Multimodal Models:**

Latest version : Llama 4 innovations introduced :

Both LLaMA and Mistral train on open source Instruction Dataset from HF – just to demonstrate that their foundation model can be instruction fine-tuned

Both also suggest that there are 3 dimensions in compute budget : model capabilities, training cost and inference cost as opposed to only the first two in Hoffmann et al. 2022. They increase the training cost by increasing data and dramatically bringing inference cost down by reducing model size!

2.3.6 Phi

The main focus is on data – better techniques for synthetic data curation

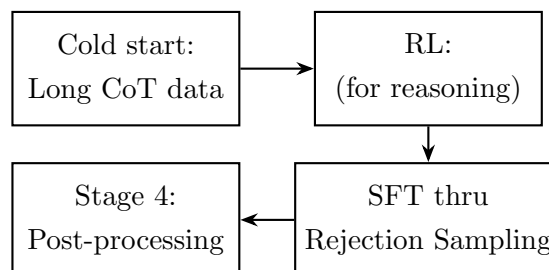
- DPO is enhanced by using token based rejection sampling

Rejection Sampling Allow the *student* model to generate several responses and evaluate responses using scripts / LLMs / Humans → train the *student* model on these refined responses

2.3.7 DeepSeek

DeepSeek-R1-Zero

DeepSeek R1:



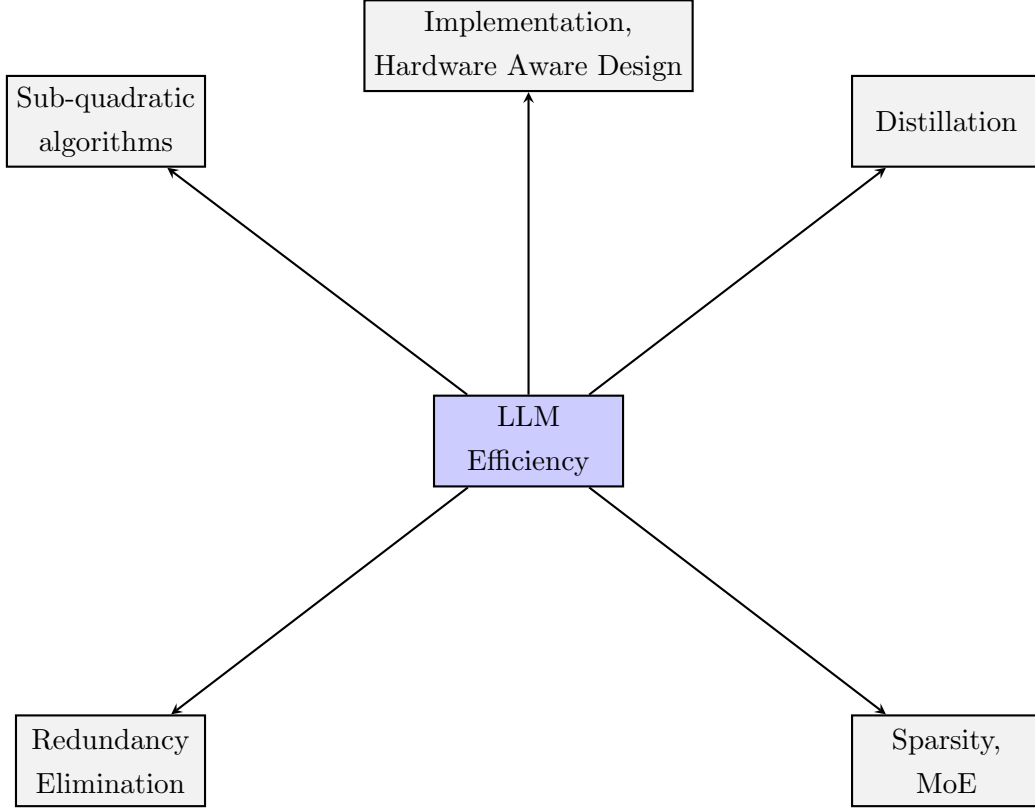
To enhance the interaction capabilities on top of DeepSeek-R1-Zero, the four stage pipeline is employed.

1. Cold-start : small amount of Long-COT data:
2. Large Scale RL for enhancing reasoning
3. Collect SFT data from CP-2 for general purpose excellence – use rejection sampling with DeepSeekV3 as the teacher for reasoning and reuse DeepSeekV3 SFT data for non-reasoning tasks
4. RL stage using reward model on Human preference data, helpfulness and harmlessness

questions: what are the key architectural innovations what are findings how they made things more efficient chat / foundation model

Note: the state of the art **Large** models like GPT-4o/5, Claude 4.0, Grok 4 are closed and little is known about the architectural developments brought into these models

Efficiency



3.1 Distillation

Introduced by Hinton, Vinyals, and Dean [2015](#) in context of classification for neural networks. Distillation is one of the ways of making training faster for smaller models by utilizing "knowledge" from Large pre-trained models. Sources : C. Yang et al. [2024](#); Y. Gu et al. [2024](#). Logit based distillation typically involves optimizing over a loss function to maximize the similarity between the teacher and student model distribution

$$\mathcal{L} = KL (p_t || p_s)$$

where p_t and p_s are the final probability distribution over the vocabulary, obtained after softmaxing the final output layer and $KL(x||y)$ is the Kullback-Leibler Divergence, defined over discrete distributions as

$$KL(x||y) = \sum_i x_i \log \frac{x_i}{y_i}$$

- Off-policy distillation : Force the output of the teacher on-to the student – the output is generated by the teacher model and the token are forced into the student model to get probability distributions, which are minimized to tune the weights $\text{Loss} = L_{off} = E_{x \sim D}[KL(P_t(y|x)||P_s(y|x))]$
- On-policy distillation : the student actively generates a response which is updated based on teacher feedback

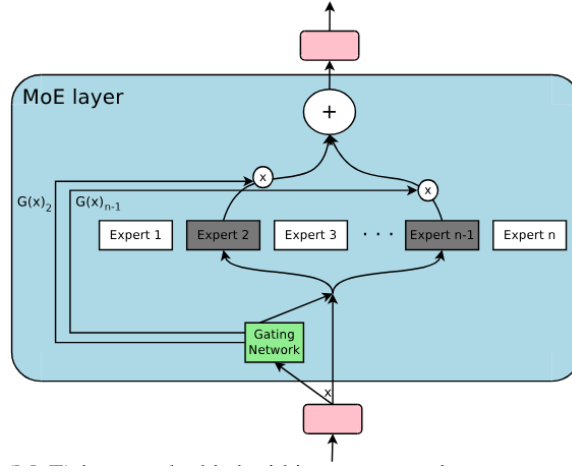


Figure 3.1: Mixture of Experts Layer

3.2 Sparsity through MOE

N. M. Shazeer et al. 2017 introduce Mixture of Experts as a way to efficiently train heavily parametrized Neural Nets by training special MOE layers containing a bunch of expert layers, some of which are used for the final computation at inference by a separate gating layer. Training is done jointly with an auxiliary loss to make sure experts are not under-utilized. See Figure 3.1 Fedus, Zoph, and N. Shazeer 2022 introduced MOE in context of Transformer based Language models to route **MLP computation** for every token to a *single* expert. Qwen Mixtral Palm

3.3 Efficient Implementation

This involves designing hardware-aware algorithms for (1) Efficient GPU utilization (2) Striking the right tradeoff between compute and memory (3) Minimizing communication latency

The most prominent set of papers along this line is the Flash-Attention series Dao et al. 2022; Dao 2024; Shah et al. 2025:

Flash-Attention 1 The key observation in the naive implementation of Transformers is I/O is a huge bottleneck over FLOPS – so the authors attempt to push FLOPS up while reducing costly HBM accesses to arrive at a tradeoff that achieves an overall speedup. Figures 3.2 and 3.3 show the difference in implementations – the key observation is to break $\mathbf{K}, \mathbf{V}, \mathbf{Q}, \mathbf{O}$ into chunks so that the intermediate matrix $\mathbf{P} \in \mathbb{R}^{N \times N}$ is never stored in and hence transferred to the HBM N is typically $\gg d$ and $\mathbf{P}$ won't fit in the SRAM. So a portion big enough to fit is computed on-chip, and its contribution is added to $\mathbf{O}$ appropriately, avoiding the costly I/O altogether! **Flash-Attention 2** improves FA by

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Figure 3.2: Standard Attention

(1) reducing non-matmul FLOPs, which are much more costly than matmul FLOPs due to NVIDIA tensor cores ; (2) Parallelizing on the sequence alongside heads and batch; (3) better warp utilization in a thread-block (set of 32 warps) as shown in Figure 3.4). Since every block of the matrix $\mathbf{V}$ has to be multiplied by every block of $\mathbf{Q}\mathbf{K}^\top$, sharing $\mathbf{V}$ across all warps minimizes communication latency

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Figure 3.3: Flash-Attention 1 implementation

and synchronization delays. **Flash-Attention 3** further improves performance specifically for H100

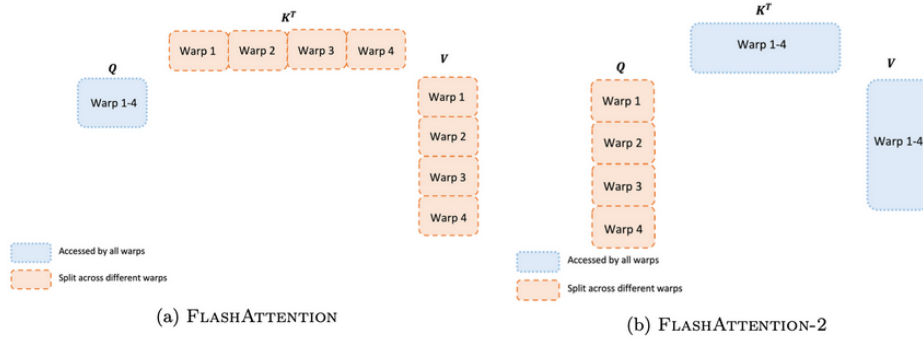


Figure 3.4: Flash Attention 2 Work Partitioning

GPUs by utilizing its tensor memory accelerator and FP-8 precision.

Paged-Attention (Kwon et al. 2023) take inspiration from virtual memory in Operating Systems - instead of the common convention of storing Pytorch tensors in contiguous space that leads to an unnecessary memory allocation (fragmentation) - they build a paging system at the granularity of KV blocks. They also support KV sharing similar to (Copy-on-Write in OS processes), across several inference requests.

3.4 Eliminating Redundancy

Frankle and Carbin 2019 propose that dense, randomly-initialized, feed-forward networks contain subnetworks (winning tickets) that—when trained in isolation— reach test accuracy comparable to the original network in a similar number of iteration. Neural networks like Transformers are heavily over-parametrized and have superflous parameters that can be eliminated. The following are some methods for efficient utilization of parameters: (// Will be elaborated upon in a later version)

3.4.1 Pruning

On the basis of pattern: structured and unstructured. On the basis of element which is eliminated:

- Width - individual weights ; attention heads ; embedding dimension (specific channels in the vector)
- Layer or depth pruning

M. Sun et al. [2024](#); Ashkboos et al. [2024](#); Ma, Fang, and X. Wang [2023](#) Practical deployment use cases of Pruning:

-

Muralidharan et al. [2024](#) describe best practices for compact LLM deployment using pruning and distillation. Llama-3.1-Minitron 4B has been pruned from LLaMA 3.1 8B. NVIDIA's NeMO framework :

3.4.2 KV-sharing and elimination

3.4.3 Quantization

Mixed-Precision Training

3.5 Some other Techniques

3.5.1 Gradient Checkpointing

3.5.2 Choosing the optimizer

Sub-quadratic algorithms have been discussed in Section [4.1](#)

Generation 3

In the entire chapter we following notation is used:

d is in the input dimension or embedding dimension

T is the length of the input sequence both during training and inference

W_α represents a weight matrix

4.1 State Space Models

State Space Models are a general class of continuous sequence to sequence models. For one dimensional sequences continuous in time, $x(t), y(t) \in \mathbb{R}$, there is a hidden state sequence $\{h\}, h(t) \in \mathbb{R}^N$.

$$\begin{aligned}\frac{dh}{dt} &= Ah(t) + Bx(t) \\ y(t) &= Ch(t)\end{aligned}$$

Since our sequences are discrete sequences, the discrete version is given by

$$\begin{aligned}h_t &= Ah_{t-1} + Bx_t \\ y_t &= Ch_t\end{aligned}\tag{4.1}$$

Notice the similarity and differences between 1.1 and 4.1 (of course there is no bias term here) – the structure is the same *but the hidden state is in a higher dimension!*, so when we represent tokens by our embedding vectors $\in \mathbb{R}^d$, h_t would $\in \mathbb{R}^{N \times d}$. This raises the dimension of A to a rank-3 tensor.

The formulation in 4.1 and formulations that look this are called **linear recurrence forms** (We will see how transformers reduce to this form!). An alternate formulation is **global convolution** :

$$\begin{aligned}h_1 &= Bx_1 \text{ assuming } h_0 = 0 \\ h_2 &= ABx_1 + Bx_2 \\ h_3 &= A^2Bx_1 + ABx_2 + Bx_3\end{aligned}$$

the output is $y_t = CA^{t-1}Bx_1 + CA^{t-2}Bx_2 + \dots + CBx_t$

Convolving $CA^{n-1}B, CA^{n-2}B, \dots, CB$ with $x_n, x_{n-1} \dots x_1$ gives the vector $(y_{2n}, y_{2n-1} \dots y_n \dots y_1)$. The sequence of our interest is $(y_n, \dots, y_1)$ Hence this is equivalent to the the convolution $K * x$ with a well defined kernel K . Note that this formulation does not hold when the parameters of the model are *time (or input) dependent*.

4.1.1 Linear Attention

Katharopoulos et al. 2020 establish the equivalence between a linear transformer (without softmax activation on the Query-Key product) and RNNs (Note that the matrices here are transposed as opposed to the original paper) Assuming X is the input sequence $X \in \mathbb{R}^{d \times T}$ and $Q, K, V \in \mathbb{R}^{d \times d}$,

the output of the Self-Attention module (unmasked) is given by :

$$\begin{aligned} Q &= W_Q X \\ K &= W_K X \\ V &= W_V X \\ A_l(X) &= V' = \text{softmax} \left(\frac{QK^T}{\sqrt{D}} \right) V. \end{aligned}$$

The softmax operation makes things tricky since it is a per-row computation that cannot be done independently before doing the matrix multiplication.

An equivalent formulation would be

$$V'_i = \frac{\sum_{j=1}^T \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^T \text{sim}(Q_i, K_j)}.$$

Thinking of sim as a kernel operation between Q_i and K_j ,

$$\begin{aligned} V'_i &= \frac{\sum_{j=1}^T \phi(Q_i)^T \phi(K_j) V_j}{\sum_{j=1}^T \phi(Q_i)^T \phi(K_j)}, \\ \Rightarrow V'_i &= \frac{\phi(Q_i)^T \sum_{j=1}^T \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^T \phi(K_j)} \end{aligned}$$

Now things become nicer, since during auto-regressive generation, what we want from the previous tokens is their keys and values. So for the i^{th} token during inference,

$$\begin{aligned} S_i &= \sum_{j=1}^i \phi(K_j) V_j^T \\ Z_i &= \sum_{j=1}^i \phi(K_j) \\ V'_i &= \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}. \end{aligned}$$

Now, S_i and Z_i can be computed from S_{i-1} and Z_{i-1} in constant time!

Also observe the stark similarity with an RNN here :

$$\begin{aligned} s_0 &= 0 \\ z_0 &= 0 \\ s_i &= s_{i-1} + \phi(W_K x_i) (W_V x_i)^T \\ z_i &= z_{i-1} + \phi(W_K x_i) \\ y_i &= f_l \left(\frac{\phi(W_Q x_i)^T s_i}{\phi(W_Q x_i)^T z_i} + x_i \right) \end{aligned}$$

During training, when the entire sequence is available we can use the mat-mul form to compute in parallel, **during inference** the complexity naturally comes down to linear, and this *gives us the best of both worlds in RNNs and Transformers!*

4.1.2 Linformer

S. Wang et al. 2020

4.1.3 RWKV

Peng et al. 2023

4.1.4 Reformer

Kitaev, Kaiser, and Levskaya 2020

4.1.5 RetNet

Y. Sun et al. 2023

4.1.6 Mamba

A. Gu and Dao 2024 Mamba is a state space model as described in 4.1 except that here the parameters A, B, C are no longer time-independent. For the input x the parameters B, C are dependent on the input according to $\text{Linear}_N(x)$. // Elaborate

The exact layer structure of the Mamba architecture is shown in Figure 4.1. The input goes through a linear transformation followed by a depth-wise one convolution given by

$$o_{i,d} = \sum_{j=0}^{k-1} W_{j,d} \cdot x_{i-j,d} + b_d$$

which followed by an activation goes through the SSM block discussed above. Finally the two projections are multiplied and brought back to the input dimension of the model.

Since the equivalence between convolution and mamba does not hold and there is no equivalent matrix kernel form as in 4.1.1, the authors propose hardware-aware algorithms by exploiting the memory hierarchy of GPUs for efficient training. Inference is linear in the sequence length and memory requirements do not scale; making Mamba excellent at handling long-context sequences. Also, note that even though LSTMs try to fit selectivity in context storage by introducing gating. Although state space models are a different formulation from RNNs but as proved by A. Gu and Dao gated RNNs are a special case of structured SSMs.

The key to deploying SSMs is the equivalent convolution formulation that helps massively parallelize training - a significant advantage over RNN-based models.

Increases Memory footprint * A note on memory footprint and IO : memory footprint increases with batchsize – this is obvious :—

4.1.7 Gated-Linear Attention

S. Yang et al. 2024

Why are these models not deployed in practice ?

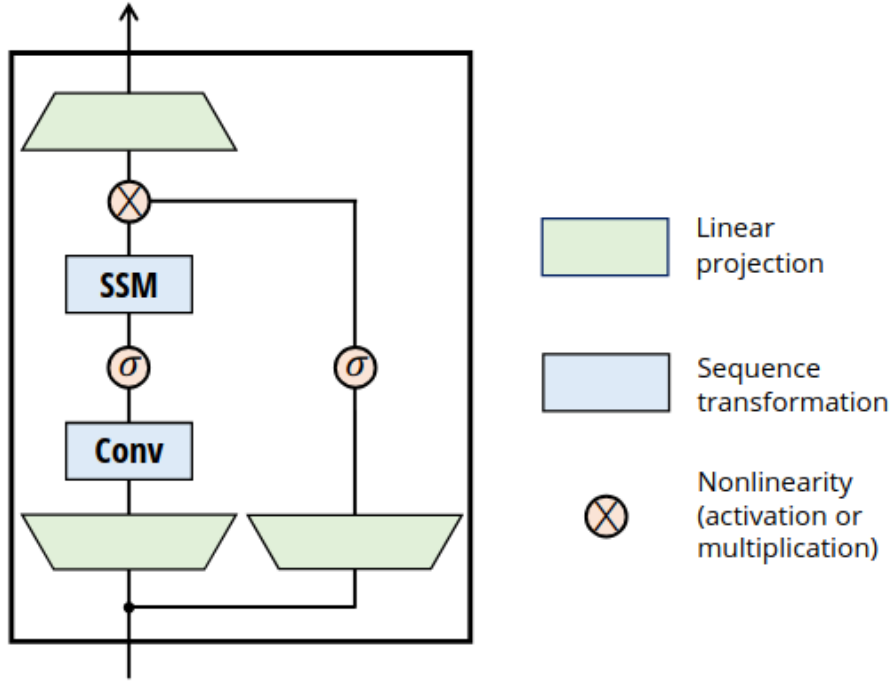


Figure 4.1: Mamba Architecture

4.2 Diffusion Language Models

As discussed in 1, the objective of language modeling is to model the distribution over token sequences, $P(t_1 \dots t_k)$. This is achieved in the auto-regressive paradigm by *iteratively* generating tokens conditioned on the context and then adding the token to the context. This paradigm is inherently sequential in the sequence dimension. Taking inspiration from diffusion models in image generation (Sohl-Dickstein et al. 2015).

Auto-regressive models are **probabilistic models** that are easier to train and sample, while diffusion or energy-based models provide much more flexibility Although sampling becomes hard. The idea is to start with noisy tokens distributed according to say a Gaussian and then *denoise* each token to model the entire sequence's probability distribution. This enables parallelism in generation.

- During training, a **forward diffusion process** introduces noise in text tokens to map them to continuous noise over t time steps, although in case of language this becomes challenging since *text representations (embeddings) are discrete vectors*¹
- For prediction during training and inference, the model applies a learnt **reverse diffusion** process to replace a subset of noisy tokens with text and over a number of time-steps the entire sequence is retrieved.

Although diffusion models provide flexibility and bidirectional context knowledge and parallel generation, Following are some disadvantages:

1. Diffusion process from a continuous to discrete space
2. Training instability
3. Computational inefficiency due to large number of denoising steps for quality in generation

¹This means that the set of embeddings that make "sense" is a finite and very sparse subset of $\mathbb{R}^{\text{vocab-size}}$, as opposed to pixel data, where intensity values are continuous. I know that IEEE floating point exists but yeah

Nie et al. 2025 Scale diffusion models to 8B params. [Gemini Diffusion](#)

Labs et al. 2025 have released a free-playground coding model at <https://www.inceptionlabs.ai/introducing-mercury>. Also utilize Transformer architecture (and hence the efficient implementations) and develop efficient kernels and inference engines specific for the diffusion process. The catch is : All machine learning models typically try to fit the data distribution by fitting model parameters on an assumed structure for the distribution. This mathematically translates to minimizing the KL divergence or maximizing the likelihood estimate between p_θ and p_{data} . The former is the distribution we arrive at, by diffusion is one paradigm for generative AI that works well for continuous image generation

Wu et al. 2025 optimize diffusion LLMs by introducing KV-Cache and Parallel Decoding similar to how Flash-Attention 1,2,3 accelerate LLMs (Dao et al. 2022; Dao 2024; Shah et al. 2025) using efficient kernel implementation and hardware-aware algorithmic modification

4.3 Energy based Models

Gladstone et al. 2025 : Inference-time computation. How to include a world-model in a multi-modal general AI ? talk about the several facets (not sure how many of them are relevant) of human cognition dynamic computation

At the first step start with a random prediction and gradient descent on the energy function till you arrive at a prediction that minimizes energy.

Energy minimization is verification and the gradient descent implicitly does generation. This model helps them generalize better to OOD data

Energy based models : learn the un-normalized probability with exponential dependence on an Energy function : $p_\theta \sim \exp(-E_B(\theta))$ "verify" the compatibility of prediction with the input

<https://atcold.github.io/NYU-DLSP20/en/week07/07-1/> **EBMs as the generalization of Diffusion Models**

So why is [Yann Le Cun](#) so adamant about energy based models? Why are the standard models called Probabilistic models ?

Assign an "energy" to every possible input-output pair – the lower this energy term, the more the "plausibility" of an output towards a given input. Formally, $E : X \times Y \rightarrow \mathbb{R}$ and the goal is to find (at inference)

$$\operatorname{argmin}_{y \in Y} E(x, y)$$

This interpretation can be translated into a probability distribution over y given x , using interconversion between energy and distribution given by Boltzmann:

$$P(y|x) = \frac{e^{-\beta F(x,y)}}{\int e^{-\beta F(x,y)} dy}$$

Training: The data points (x, y) are the minimum energy regions of the Energy landscape. To make

sure that the prediction is correct; make sure that $\hat{y}$ corresponds to the lowest energy. Two methods:

- Contrastive Learning : Push negative sample energy up and positive sample energy down – inefficient
- Regularized Learning (// to elaborate)

Question : How do you model uncertainty in predictions ? Have same energy for multiple predictions ?

Note that maximum likelihood estimation is contrastive but makes things unsmooth! The key question though is "How do you design energy functions ? "

What challenges does this possibly solve ?

- Too much data
- Task agnostic reasoning
- Multimodality
- System 2 thinking alignment in language (NLP)
- Uncertainty (multiple outputs) and dynamic computation even for non-reasoning problems like language modeling and text classification

Human learning is known to be non-supervised or a wild combination of all of them ? Humans also learn world models in parallel. Reasoning capabilities are mostly intrinsic. Humans are dynamical systems – not a big deal can simulate a human life very quickly on a GPU

How is training done ? Transformers can inherently not solve all problems in the P class – fixed depth leads to the class AC^0 or TC^0 . But even as humans we are not particularly interested in solving P class problems - most of the problems we regard as reasoning problems are solvable quickly – the problem with coming up with programs is the heavy translation into abstract representation and which is why we resort to techniques in data science

4.4 Hyena architectures

Paper : convolutions in language models: <https://arxiv.org/pdf/2302.10866>

Faster scaling but heavy compute requirements Verification over prediction

4.5 Hybrid Models

Hybird models aim to combine State-Space models and Quadratic transformer blocks into the same model (Some models also combine Diffusion and auto-regressive models) Some works in these direction include:

- Convolutional Hybrid Models: Ku et al. [2025](#)
- Dual-State Linear Attention Ro et al. [2025](#)
- LLaMA-ZEBRA M. Yang et al. [2025](#)

Transformer++ refers to a consolidated training recipe for feed-forward Transformers (decoder-only for AR LM) with up-to-date implementation choices, used as the reference system in recent literature (Mamba popularized)

4.6 Notion of Heirarchical Thinking

Papers that talk about this:

- HRM - solves the problem in early convergence through :
can we use regularized energy landscape for better convergence - not an energy model but some inspirations from Energy based models ?
- EBT(Gladstone et al. [2025](#))

4.7 Scaling laws for Language models

Kaplan et al. [2020](#) Hoffmann et al. [2022](#)

Scaling experiments:

Does AdamW hyperparameter $\beta = (.9, .95)$ for smaller models help ?

Part II

LLMs as Chat-bots

Post-training

Foundation model like GPT, Claude and Gemini and language generation models and cannot simulate conversations well – well in the sense that they are not *meant* for conversation - they are meant for "yapping" given some prefill initialization (prompt as a more fancy term)

A typical pipeline in deploying production grade "chat / conversation" models, starting from the "foundation" language generation models is shown below:

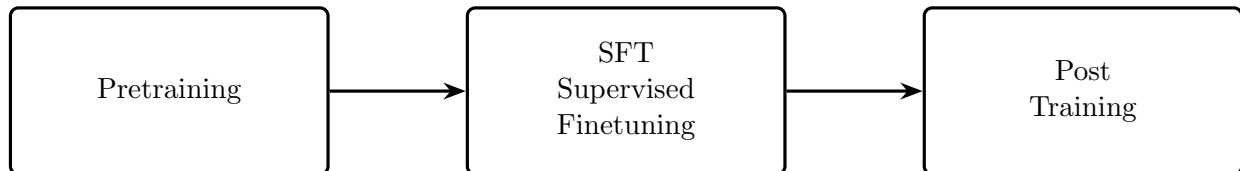


Figure 5.1: Training pipeline.

5.1 RLHF : Reinforcement Learning from Human Feedback

- Integrate stylistic and behavioural features into language models.

Preference Learning : – human annotations starting from the earlier versions of GPT → ChatGPT → GPT-o1

5.2 Instruction Fine-tuning

This step enables models to respond in a question-answer format and is done by training the foundation model on a small sample of conversations

LLaVA (H. Liu et al. [2023](#)) follow a visual instruction fine-tuning. Reward systems for LLM RL include optimizing upon:

- Preference alignment : tone and style of the response
- format alignment – format of the training or custom
- instruction alignment : user expected response and correct instruction interpretation
- agent ability : to invoke tools

Reward systems can be based on rule-based rewards – good for math and coding tasks or direct-output based rewards

For more flexibility to *learn* a highly complex reward function from human preferences, a reward model is used. This model then scores the LLM's response on a given prompt. Note that is NOT agentic interaction. Exact Reward systems :

- **Rule based rewards:** prompt the model, get its response and match it using *deterministic* programs (script/regex) – gives sharp unambiguity and correctness

- **Model based rewards with ref answer** : A *teacher* model evaluates the responses of the student against reference responses – more flexibility and suitable for language and conversation
- **Model-based Reward without Reference Answer** A reward model (NOT AN LLM) trained on human preference data scores answers without a reference – to make the model even more flexible

Distillation turns out to be much more efficient and performant as compared to direct RL for small models A. Yang et al. [2025](#); DeepSeek-AI et al. [2025](#)

AGI and Reasoning

Large Language Models are by themselves not so useful – conversational agents are obtained by fine-tuning foundational models on "instruction-fine-tuning"

Some post-training methods:

- **SFT** : Asking the model to generate multiple responses on the same prompt - evaluating using another LLM, training objective for the best response
- **RLHF**
- **DPO** - Rafailov et al. [2023](#) : to align the model with human preferences, and also to steer the model away from unwanted behavior through pairs of desired and undesired outputs
- **GRPO** - Shao et al. [2024](#) : Group Relative policy optimization

Other parts of typical LRM training pipeline

- Structure Search
-

6.1 How are reasoning models like GPT-o, Claude Thinking etc trained?

What makes them gain $> 0\%$ performance on ARC-ARG1/2/3 ?

6.1.1 GPT-o series

o1 scaled the CoT length to enhance the reasoning capabilities of GPT, but this comes at a significant inference-time cost DeepSeek came up with pure RL to mitigate this issue

6.2 ARC-AGI Benchmarks

6.3 Some other interesting benchmarks

Pass@X score on AIME : at least one of X responses generated by the model is correct **Maj@X** : majority of the answers produced by the model is correct.

6.4 Current Reasoning Landscape

Early ideas included: Scratchpads and intermediate computations: "Show Your Work: Scratchpads for intermediate computation" introduced emitting intermediate steps to a "scratchpad,"

Brown et al. showed that GPT-3 could learn from few examples of a given task - early demonstration in reasoning capabilities in LLMs. *In-context learning* abilities of LLMs - learning at inference time

purely by giving examples and no gradient update.

Wei et al. showed that can enhance model's reasoning capabilities by increasing performance on GSM-8K. Owing to few-shot exemplar learning in context, they tried to append 8 examples of $\langle \text{question} \rangle \langle \text{thought} \rangle \langle \text{answer} \rangle$; then asked the model the actual question for each sample in the benchmark asking the model to follow similar reasoning procedures. This prompting itself showed increased reasoning abilities in a foundational LM.

How is CoT included in the training of foundation models ?

Zero-shot : Directly ask the question to the model

Chain-of-thought prompt : elaborate the reasoning steps to arrive at the answer.

Yao et al. generalize linear CoT reasoning to a non-linear -

6.5 Hierarchical Reasoning Model

- Solve early convergence in RNN type models by forcing the L-module to arrive at a local equilibrium $\rightarrow$ force a new high level state by updating the H-module; and then forcing L to a new local eq
- Monitored NT steps made stable and deep? – dynamic computation through multiple forward passes

Neuro Symbolic based reasoning. (what is integrated neuro symbolic Reasoning) **<empty citation>**
AlphaGeometry demonstrated SOTA in solving IMO geometry problems - neural language model and symbolic deduction engine

Synthetic data generation : Construct 1 billion examples of random math figures and then programmatically find all relations – use an automated theorem prover to get to a proof using given facts and then models gives constructs to arrive at the proof with just the diagram to arrive at a proof.

Verification in LLM Can this somehow relate to Verification of LLM responses ? Self-consistency improves CoT : X. Wang et al. 2023 Energy Based Transformers : Gladstone et al. 2025 Qwen with questions : QwQ

The idea of system 1 and 2 thinking from Thinking Fast and Slow is being prominently deployed - what if we take this to several hierarchies ?

6.6 Scientist Language Models

FunSearch: solving (?) open problems in math using LLMs

Early formal logic based reasoning systems : Symbolic logic systems

- Prolog and LISP based automated reasoning systems
- Define facts, rules and reason over random queries through propositional and predicate logic
- Drawback is conversion of natural language to structured boolean systems

Why have we not reached AGI yet, despite models being able to solve IMO problems?

The objective despite introducing reasoning is of Pattern Recognition from data

Can we make these models less robust by giving them access to less data?

Why is generalization a problem - because of lack of exposure to real-time environments ?

6.7 Monte Carlo Search Method

Key idea is to do exploration and exploitation in a tree like structure with child node selection incentivized appropriately (UCB1 formula) More of a motion-planning and game thing for exploring possible reasoning paths. Did not work well for DeepSeek-R1

What are the things to consider :

Long term planning

Dynamic reasoning

Multi-Step inference problems

computational depth

6.8 Reinforcement Learning

Is RL the basis for human learning? Limbic reward mechanisms in Humans : **Q-learning and Deep Q networks** AlphaZero and AlphaGo used deep learning for high-dimensional and non-linear decision dynamics; integrated with MCTS

Move-37 Emergent strategies : backtracking, self-verification and new ways in general

Language can be learned through data-oriented practice (if we want to simulate human learning) – this can be modeled as a reward based RL to signal direction ?

Where and where not is RL needed ? : RL will work where the notion of reward post verification exists – math and coding are good examples 5th plan

The notion of a verifier Verification is a strong element of human cognition ? Gladstone et al. [2025](#)

- Exploration and structure are two conflicting goals ?
- Overthinking
- Replacing simple CoT with Slow-Thinking SFT data
- Training LLMs for slow-thinking, as characterized by the LongCoT approach, results in relatively uniform gradient norms across different layers. In contrast, fast-thinking, exemplified by the simplified CoT method, generates larger gradient magnitudes in the earlier layers, along with significant variability in gradient norms across layers
- Lack of an internal world-model
- These are well applied to ultra-complicated and specific tasks - Math, Coding and Medical prediction

Monte-Carlo Tree Search

Backpropagation for predicting future rewards

CoT is a single-pass method - linear

MCTS ; ToT – tree structure for exploring reasoning paths – but MCTS has backward feedback CoT and ToT are prompting frameworks. MCTS is dynamic in the sense that final reward is backpropagated upwards all the nodes leading to that path updating their individual rewards.

How is ToT different from MCTS ?

MCTS is iterative. Multimodal reasoning ?

Is complicating the reasoning process complicating things in general ? Challenge : CoT and long reasoning methods fall into Recursive Reasoning Loops Models which perform great at math and coding are bad at NLU and NLG somehow – Qwen has the problem of language mixing Integrating MCTS with LLMs. Error correction and Self-evaluation ; Verification

6.9 Latent Reasoning

The field is pretty early and some papers in this direction include:

1. <https://arxiv.org/pdf/2412.06769>
2. <https://arxiv.org/pdf/2502.17416>
3. <https://openreview.net/pdf?id=D6o6Bwtq7h>

Safety in AI

Safety on various facets:

1. Data Leakage:
 - (a) Client data extraction through prompts in LLMs
 - (b) Server data extraction through prompts
 - (c) Federated Learning : gradient reconstruction and data-set confidence matching
2. Jail-breaking and toxic/inappropriate responses
3. Hallucination
4. Alignment
5. Bias problem : pretty much solved in the early days after commercial GPT

Interpretability insights

Token representation Redundancy of model units - pathways, weights, layers ? Explainability

8.1 Random Stuff

In context learning

8.2 Optimizers

8.3 Transformer++ recipe

RoPE encoding and SwiGLU MLP

8.4 Datasets

Note that the number of tokens is Tokenizer-dependent

- [Pile](#) Gao et al. [2020](#) $\approx$ 300 B (using GPT tokenizer) - shorter version is MiniPile $\approx$ 1.6 B
- [RedPajama](#) - v2 has $\approx$ 30T tokens - SlimPajama 672 B
- Wikitext103 / 2 – usually used for measuring perplexity on wikipedia statements
- C4 - 175 B
-
-

8.5 benchmarks

- Hellaswag Zellers et al. [2019](#),
- OpenBookQA Mihaylov et al. [2018](#),
- WinoGrande Sakaguchi et al. [2021](#),
- ARC-Easy and ARC-Challenge P. Clark et al. [2018](#)
- CommonsenseQA Talmor et al. [2019](#)
- BoolQ C. Clark et al. [2019](#)
- PIQA Bisk et al. [2020](#)
- SIQA Sap et al. [2019](#)
- MMLU Hendrycks et al. [2021](#)
- GSM-8K Cobbe et al. [2021](#)

- TriviaQA Joshi et al. 2017
- RACE Zheng et al. 2024
- Natural-QA Kwiatkowski et al. 2019
- MT-Bench
- Alpaca Eval
- Humaneval
- MBPP
- BBH (n-shot)
- AGI Eval
- QUAC Choi et al. 2018

Some small datasets to measure perplexity on :

- Wikitext-2/103 Merity et al. 2017
- Lambada

These evaluations can be easily done using EleutherAI's LM EvalHarness framework **eval-harness**. some other terminology ROPE : A. Gu and Dao find that this works much better than vanilla positional encoding shared embeddings Elmo is a deep-contextualized word-representation derived from a bidirectional language model

A. Yang et al. 2025 observe that for smaller models, distillation from Large teacher models highly outperforms RL methods

I believe in the following two theories of how computation is happening inside the transformer architecture:

8.6 Theory of consistent information flow

<https://arxiv.org/pdf/2409.03621> say that tampering with one layer's hidden states by replacement and shuffling does not affect the final prediction – but this analysis is very limited on the kind of evaluations have been done. Will this still hold in tasks with very complicated reasoning ? The other thing is number of tampering ... how robust is the model ?

The primitive assumption is there is layer-wise separation of the contribution to the final predicted token Now some potential theories:

1. Information flow - the model starts with consolidating consistent information and that tampering at a later layer is a small addition to a large vector. If that were the case then early tampering would distangle some information(?) and then further layers should have fixed it ? An explanation : later layers attention-wise feebly contribute and nothing changes - confirming the hypothesis
How about tampering at a layer and then replacing the attention weights from layer 0 to n-k?
– may confirm the reconstruction / large contribution hypothesis → makes sense because of

residual stream If attention contributions are really feeble then why not just set them to zero or random for a large number of layers at the top ? Another thing could be reducing the magnitude of contribution with the same magnitude ratio to the stream!!

2. 8.7 Theory of iterative inference

Shows that ResNet computation is gradient descent in the activation space but no theoretical backing is given as to "why" that happens

Each individual module is performing "locally" similar computation - if the initial layers perform representation (feature) refinement and the later layers try to map it to predictions then dropping attention later on would make sense

"where to start" is an experimental platonification

Multimodality - Transformers as efficient parametrizers

- want to learn about diffusion and generation in other modalities - want to test out batch stuff and better understanding of GPU - write a thesis on training LMs - want to learn the usage of transformers elsewhere

Bibliography

- Ainslie, J. et al. (Dec. 2023). “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by H. Bouamor, J. Pino, and K. Bali. Singapore: Association for Computational Linguistics, pp. 4895–4901. DOI: [10.18653/v1/2023.emnlp-main.298](https://doi.org/10.18653/v1/2023.emnlp-main.298). URL: <https://aclanthology.org/2023.emnlp-main.298/>.
- Ashkboos, S. et al. (2024). “SliceGPT: Compress Large Language Models by Deleting Rows and Columns”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=vXxardq6db>.
- Biderman, S. et al. (2023). “Pythia: a suite for analyzing large language models across training and scaling”. In: *Proceedings of the 40th International Conference on Machine Learning*. ICML’23. Honolulu, Hawaii, USA: JMLR.org.
- Bisk, Y. et al. (Apr. 2020). “PIQA: Reasoning about Physical Commonsense in Natural Language”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.05, pp. 7432–7439. DOI: [10.1609/aaai.v34i05.6239](https://doi.org/10.1609/aaai.v34i05.6239). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- Brown, T. B. et al. (2020). “Language models are few-shot learners”. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS ’20. Vancouver, BC, Canada: Curran Associates Inc. ISBN: 9781713829546.
- Choi, E. et al. (Oct. 2018). “QuAC: Question Answering in Context”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, pp. 2174–2184. DOI: [10.18653/v1/D18-1241](https://doi.org/10.18653/v1/D18-1241).
- Clark, C. et al. (June 2019). “BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 2924–2936. DOI: [10.18653/v1/N19-1300](https://doi.org/10.18653/v1/N19-1300). URL: <https://aclanthology.org/N19-1300/>.
- Clark, P. et al. (2018). *Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge*. arXiv: [1803.05457](https://arxiv.org/abs/1803.05457) [cs.AI]. URL: <https://arxiv.org/abs/1803.05457>.
- Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv: [2110.14168](https://arxiv.org/abs/2110.14168) [cs.LG]. URL: <https://arxiv.org/abs/2110.14168>.
- Dao, T. (2024). “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- Dao, T. et al. (2022). “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., pp. 16344–16359. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.
- DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv: [2501.12948](https://arxiv.org/abs/2501.12948) [cs.CL]. URL: <https://arxiv.org/abs/2501.12948>.
- Devlin, J. et al. (June 2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the*

- Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). URL: <https://aclanthology.org/N19-1423/>.
- Fedus, W., B. Zoph, and N. Shazeer (Jan. 2022). “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity”. In: *J. Mach. Learn. Res.* 23.1. ISSN: 1532-4435.
- Frankle, J. and M. Carbin (2019). *The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks*. arXiv: [1803.03635 \[cs.LG\]](https://arxiv.org/abs/1803.03635). URL: <https://arxiv.org/abs/1803.03635>.
- Gao, L. et al. (2020). *The Pile: An 800GB Dataset of Diverse Text for Language Modeling*. arXiv: [2101.00027 \[cs.CL\]](https://arxiv.org/abs/2101.00027). URL: <https://arxiv.org/abs/2101.00027>.
- Gladstone, A. et al. (2025). *Energy-Based Transformers are Scalable Learners and Thinkers*. arXiv: [2507.02092 \[cs.LG\]](https://arxiv.org/abs/2507.02092). URL: <https://arxiv.org/abs/2507.02092>.
- Gu, A. and T. Dao (2024). “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *First Conference on Language Modeling*. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- Gu, Y. et al. (2024). “MiniLLM: Knowledge Distillation of Large Language Models”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=5h0qf7IBZZ>.
- Hendrycks, D. et al. (2021). “Measuring Massive Multitask Language Understanding”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Hinton, G. E., O. Vinyals, and J. Dean (2015). “Distilling the Knowledge in a Neural Network”. In: *ArXiv abs/1503.02531*. URL: <https://api.semanticscholar.org/CorpusID:7200347>.
- Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv: [2203.15556 \[cs.CL\]](https://arxiv.org/abs/2203.15556). URL: <https://arxiv.org/abs/2203.15556>.
- Joshi, M. et al. (July 2017). “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, pp. 1601–1611. DOI: [10.18653/v1/P17-1147](https://doi.org/10.18653/v1/P17-1147).
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv: [2001.08361 \[cs.LG\]](https://arxiv.org/abs/2001.08361). URL: <https://arxiv.org/abs/2001.08361>.
- Katharopoulos, A. et al. (2020). “Transformers are RNNs: fast autoregressive transformers with linear attention”. In: *Proceedings of the 37th International Conference on Machine Learning. ICML’20*. JMLR.org.
- Kitaev, N., L. Kaiser, and A. Levskaya (2020). “Reformer: The Efficient Transformer”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=rkgNKkHtvB>.
- Ku, J. et al. (2025). *Systems and Algorithms for Convolutional Multi-Hybrid Language Models at Scale*. arXiv: [2503.01868 \[cs.LG\]](https://arxiv.org/abs/2503.01868). URL: <https://arxiv.org/abs/2503.01868>.
- Kwiatkowski, T. et al. (2019). “Natural Questions: A Benchmark for Question Answering Research”. In: vol. 7, pp. 453–466. DOI: [10.1162/tac1_a_00276](https://doi.org/10.1162/tac1_a_00276).
- Kwon, W. et al. (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *SOSP ’23*. Koblenz, Germany: Association for Computing Machinery, pp. 611–626. ISBN: 9798400702297. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165). URL: <https://doi.org/10.1145/3600006.3613165>.
- Labs, I. et al. (2025). *Mercury: Ultra-Fast Language Models Based on Diffusion*. arXiv: [2506.17298 \[cs.CL\]](https://arxiv.org/abs/2506.17298). URL: <https://arxiv.org/abs/2506.17298>.
- Liu, H. et al. (2023). “Visual Instruction Tuning”. In: *NeurIPS*.

- Liu, Y. et al. (2019). “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. In: *ArXiv* abs/1907.11692. URL: <https://api.semanticscholar.org/CorpusID:198953378>.
- Liu, Z. et al. (2024). “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases”. In: *Forty-first International Conference on Machine Learning*. URL: <https://openreview.net/forum?id=EIGbXbxcUQ>.
- Ma, X., G. Fang, and X. Wang (2023). “LLM-Pruner: On the Structural Pruning of Large Language Models”. In: *Thirty-seventh Conference on Neural Information Processing Systems*. URL: <https://openreview.net/forum?id=J8Ajf9WfXP>.
- Merity, S. et al. (2017). “Pointer Sentinel Mixture Models”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Byj72udxe>.
- Mihaylov, T. et al. (Oct. 2018). “Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Ed. by E. Riloff et al. Brussels, Belgium: Association for Computational Linguistics, pp. 2381–2391. DOI: [10.18653/v1/D18-1260](https://doi.org/10.18653/v1/D18-1260). URL: <https://aclanthology.org/D18-1260/>.
- Muralidharan, S. et al. (2024). “Compact Language Models via Pruning and Knowledge Distillation”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. URL: <https://openreview.net/forum?id=9U0nLnNMJ7>.
- Nie, S. et al. (2025). *Large Language Diffusion Models*. arXiv: [2502.09992](https://arxiv.org/abs/2502.09992) [cs.CL]. URL: <https://arxiv.org/abs/2502.09992>.
- Peng, B. et al. (Dec. 2023). “RWKV: Reinventing RNNs for the Transformer Era”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Ed. by H. Bouamor, J. Pino, and K. Bali. Singapore: Association for Computational Linguistics, pp. 14048–14077. DOI: [10.18653/v1/2023.findings-emnlp.936](https://doi.org/10.18653/v1/2023.findings-emnlp.936). URL: <https://aclanthology.org/2023.findings-emnlp.936/>.
- Radford, A. and K. Narasimhan (2018). “Improving Language Understanding by Generative Pre-Training”. In: URL: <https://api.semanticscholar.org/CorpusID:49313245>.
- Rafailov, R. et al. (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Thirty-seventh Conference on Neural Information Processing Systems*. URL: <https://openreview.net/forum?id=HPuSIXJaa9>.
- Ro, Y. et al. (2025). “On-the-Fly Adaptive Distillation of Transformer to Dual-State Linear Attention for Long-Context LLM Serving”. In: *Forty-second International Conference on Machine Learning*. URL: <https://openreview.net/forum?id=pqHWzviKKN>.
- Sakaguchi, K. et al. (Aug. 2021). “WinoGrande: an adversarial winograd schema challenge at scale”. In: *Commun. ACM* 64.9, pp. 99–106. ISSN: 0001-0782. DOI: [10.1145/3474381](https://doi.org/10.1145/3474381). URL: <https://doi.org/10.1145/3474381>.
- Sap, M. et al. (Nov. 2019). “Social IQa: Commonsense Reasoning about Social Interactions”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by K. Inui et al. Hong Kong, China: Association for Computational Linguistics, pp. 4463–4473. DOI: [10.18653/v1/D19-1454](https://doi.org/10.18653/v1/D19-1454). URL: <https://aclanthology.org/D19-1454/>.
- Shah, J. et al. (2025). “FlashAttention-3: fast and accurate attention with asynchrony and low-precision”. In: *Proceedings of the 38th International Conference on Neural Information Processing Systems*. NIPS ’24. Vancouver, BC, Canada: Curran Associates Inc. ISBN: 9798331314385.
- Shao, Z. et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv: [2402.03300](https://arxiv.org/abs/2402.03300) [cs.CL]. URL: <https://arxiv.org/abs/2402.03300>.

- Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv: 1911.02150 [cs.NE]. URL: <https://arxiv.org/abs/1911.02150>.
- Shazeer, N. M. et al. (2017). “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *ArXiv abs/1701.06538*. URL: <https://api.semanticscholar.org/CorpusID:12462234>.
- Sohl-Dickstein, J. et al. (July 2015). “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by F. Bach and D. Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, pp. 2256–2265. URL: <https://proceedings.mlr.press/v37/sohl-dickstein15.html>.
- Sun, M. et al. (2024). “A Simple and Effective Pruning Approach for Large Language Models”. In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=PxoFut3dWW>.
- Sun, Y. et al. (2023). *Retentive Network: A Successor to Transformer for Large Language Models*. arXiv: 2307.08621 [cs.CL]. URL: <https://arxiv.org/abs/2307.08621>.
- Talmor, A. et al. (June 2019). “CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 4149–4158. DOI: 10.18653/v1/N19-1421.
- Vaswani, A. et al. (2017). “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Wang, S. et al. (2020). *Linformer: Self-Attention with Linear Complexity*. arXiv: 2006.04768 [cs.LG]. URL: <https://arxiv.org/abs/2006.04768>.
- Wang, X. et al. (2023). “Self-Consistency Improves Chain of Thought Reasoning in Language Models”. In: *The Eleventh International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=1PL1NIMMrw>.
- Wei, J. et al. (2022). “Chain-of-thought prompting elicits reasoning in large language models”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. New Orleans, LA, USA: Curran Associates Inc. ISBN: 9781713871088.
- Wu, C. et al. (2025). *Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding*. arXiv: 2505.22618 [cs.CL]. URL: <https://arxiv.org/abs/2505.22618>.
- Yang, A. et al. (2025). *Qwen3 Technical Report*. arXiv: 2505.09388 [cs.CL]. URL: <https://arxiv.org/abs/2505.09388>.
- Yang, C. et al. (Oct. 2024). “Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application”. In: *ACM Trans. Intell. Syst. Technol.* ISSN: 2157-6904. DOI: 10.1145/3699518. URL: <https://doi.org/10.1145/3699518>.
- Yang, M. et al. (May 2025). “Zebra-Llama: Towards Extremely Efficient Hybrid Models”. In: *CoRR abs/2505.17272*. URL: <https://doi.org/10.48550/arXiv.2505.17272>.
- Yang, S. et al. (2024). “Gated linear attention transformers with hardware-efficient training”. In: *Proceedings of the 41st International Conference on Machine Learning*. ICML’24. Vienna, Austria: JMLR.org.
- Yao, S. et al. (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., pp. 11809–11822. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf.

- Zellers, R. et al. (July 2019). “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by A. Korhonen, D. Traum, and L. Màrquez. Florence, Italy: Association for Computational Linguistics, pp. 4791–4800. DOI: [10.18653/v1/P19-1472](https://doi.org/10.18653/v1/P19-1472). URL: <https://aclanthology.org/P19-1472/>.
- Zheng, J. et al. (2024). *Beyond Correctness: Benchmarking Multi-dimensional Code Generation for Large Language Models*. arXiv: [2407.11470](https://arxiv.org/abs/2407.11470) [cs.SE]. URL: <https://arxiv.org/abs/2407.11470>.